TP N.º 8 — Food Store en la feria: base de la tablet y cierre de la capa de datos

BD II (UTN) · Motores: PostgreSQL 16+ (`food_store_tp3`, copia), SQLite 3.37+ (tablet), MongoDB (U7). Regla: **el móvil propone, el servidor dispone**. La tablet solo consulta y arma borradores; precio, stock y ventas los decide PostgreSQL. `SIMULADO_APROBADO` **no** es una venta. Datos inventados, sin credenciales reales.

Repositorio: `https://github.com/matiiiazc/db_food_store` · **Integrantes:** *(completar)*

Evidencias: salidas de ejecución reales (archivos `movil/evidencia_*.txt`) resumidas en este informe.

1.a — Qué datos viajan a la tablet

Campo de producto	¿Viaja?	Motivo
`id`, `nombre`	Sí	Identificar y mostrar el producto en el catálogo offline.
`precio` → `precio_centavos` (INTEGER)	Sí	Mostrar precio y armar el borrador; va en centavos (sin flotantes). Es el precio *visto*, no el autorizado.
`stock` → `stock_visto`	Sí	Orientar la carga del borrador (el stock real lo valida el servidor).
`disponible`, `eliminado` (flags 0/1)	Sí	Filtrar el catálogo (baja lógica) y detectar rechazos.
`fecha_copia`	Sí (metadato local)	Trazar la antigüedad de la instantánea.
Credenciales/usuarios (`usuario.email`, `password_hash`), roles, datos personales	**No**	Sensibles; la tablet no autentica contra la base maestra ni gestiona cuentas.
Pedidos/ventas confirmadas, detalle de pago sensible	**No**	La venta y el stock los decide/confirma PostgreSQL; la tablet solo lleva borradores y cola de sync.

Viajan solo los datos mínimos para **consultar catálogo y armar borradores** offline. La **fuente de verdad** es **PostgreSQL** (precios, stock, disponibilidad, ventas); la tablet trabaja sobre una **instantánea/caché** y sus decisiones se resuelven contra el servidor (semáforo), nunca al revés.

2.c — Casos C2 a C6 (simulación, sin modificar PostgreSQL)

Cambios hechos sobre la **copia/instantánea** (`snapshot_servidor`) en SQLite y nueva «exportación» simulada; semáforo = `movil/semaforo.sql` (solo PENDIENTE). Evidencia: `movil/evidencia_casos.txt`. Precios base (centavos): prod 1=124984, 2=308614, 3=179073, 5=493511, 6=275343.

Caso	Comando / preparación	Resultado	Explicación
C2	Borrador prod 1×2 @124984 visto. Instantánea nueva: prod 1 → 129984. Correr semáforo. Cliente acepta: `UPDATE item_borrador` precio=129984; `INSERT` nueva solicitud (PENDIENTE); semáforo.	1ª: `('C2-sol-0001','…','CONFLICTO','precio cambiado')`. 2ª (nueva sol.): `('C2-sol-0002','…','SIMULADO_APROBADO','todo coincide')`.	Precio actual distinto del visto, sin otros problemas ⇒ CONFLICTO. Al aceptar y re-solicitar con precio actualizado ⇒ SIMULADO_APROBADO.
C3	Borrador prod 1×200 @124984. Instantánea: prod 1 stock=50. Semáforo.	`('C3-sol-0001','…','RECHAZADO','stock insuficiente')`.	Cantidad (200) > stock (50) ⇒ RECHAZADO (rojo).
C4	Borrador prod 6×1 @275343. Instantánea: prod 6 `eliminado=1, disponible=0`. Semáforo. Recargar `producto_cache` desde snapshot.	`('C4-sol-0001','…','RECHAZADO','producto no disponible o dado de baja')`. Catálogo disponible excluye prod 6; fila en cache: `(6,0,1)` (queda).	Producto de baja ⇒ RECHAZADO. Al recargar, no aparece como disponible pero la fila permanece (baja lógica).
C5	Borrador prod 2×1 @308614, prod 3×2 @179073, coincidente con instantánea. Semáforo.	`('C5-sol-0001','…','SIMULADO_APROBADO','todo coincide con el servidor')`.	Sin diferencias de precio ni problemas de stock/disponibilidad ⇒ verde.
C6	Borrador+solicitud `C6-sol-0001` → semáforo (resuelta). Reinsertar **mismo** `solicitud_uuid`. Cambiar precio en snapshot y repetir semáforo.	1ª: `SIMULADO_APROBADO`. Reenvío UUID: `ERROR: UNIQUE constraint failed: cola_sync.solicitud_uuid`. Tras repetir: sigue `SIMULADO_APROBADO`.	UUID duplicado falla por PK (no se duplica) y el semáforo solo evalúa PENDIENTE, así que no reevalúa. **Protección real (parada 8):** idempotencia en el *servidor* con `solicitud_uuid` como clave de idempotencia + restricción única/transacción al confirmar la venta, para que un reintento de red devuelva el mismo resultado sin duplicar pedidos ni afectar stock.

PostgreSQL no cambió (consulta antes/después, idéntica): `pedido`=200401, `detalle_pedido`=601205; muestreo `producto` (id 1,2,3,5,6) con los mismos precio/stock/activo (`evidencia_pg_antes.txt` = `evidencia_pg_despues.txt`).

3.a — Mapa de la capa de datos (tres motores)

Motor	Qué guarda	Quién manda (fuente de verdad)
PostgreSQL (`food_store_tp3`)	Modelo relacional transaccional: `producto`, `categoria`, `cliente`, `pedido`/`detalle_pedido`, `usuario`/auditoría, reglas (FK/CHECK/triggers/SP), índices/vistas (p.ej. `v_snapshot_movil`).	**Manda sobre todo el dato operativo y normativo**: precios, stock, disponibilidad, ventas/pedidos confirmados, integridad y seguridad. Es la **fuente de verdad**.
MongoDB (Unidad 7, `food_store_nosql.pedidos`)	Vista documental/lectura de pedidos (agregado embebido: cliente + líneas con precios históricos), migrada desde PostgreSQL para consulta de pedido completo.	**No manda**: es **copia de lectura** derivada. La verdad de pedidos, precios históricos vigentes y consistencia sigue en PostgreSQL; Mongo se usa como proyección complementaria (lectura/agregado).
SQLite (tablet, `foodstore_movil.db`)	Caché/instantánea de catálogo (`producto_cache`/`snapshot_servidor`), borradores locales (`pedido_borrador`/`item_borrador`) y cola de sincronización (`cola_sync`) con semáforo.	**No manda**: **propone** (borradores/solicitudes). Nunca cambia stock/precio/ventas del servidor; los estados son simulados hasta decisión del servidor. Fuente de verdad de lo que muestra: PostgreSQL (vía snapshot).

3.b — Plan de mantenimiento (agregado al de la Unidad 6)

- **Respaldo y prueba de restauración:** respaldar PostgreSQL (base completa `food_store_tp3`) con `pg_dump` (lógico, periódico) + WAL/archiving para PITR según criticidad; respaldar también la configuración (roles/permisos). **Probar la restauración** regularmente en un entorno de pruebas: restaurar el dump/PITR a una base aparte y verificar conteos, integridad (FK/CHECK), objetos clave y consultas de negocio (p.ej. totales de pedido/detalle, muestreo de stock/precio) antes de dar el respaldo por válido.
- **Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets?** La verdad del servidor «retrocede», pero las tablets conservan caché/borradores/solicitudes basados en instantáneas posteriores. Al reconectar, deben **invalidar/descartar la caché local y recargar una instantánea nueva** desde el PostgreSQL restaurado; las solicitudes `PENDIENTE`/resueltas se **re-evalúan contra la nueva instantánea** (pueden pasar a RECHAZADO/CONFLICTO) y **nunca** se aplican cambios locales al servidor automáticamente. Los borradores con conflictos se resuelven con el cliente. (Idempotencia por `solicitud_uuid` en servidor evita duplicados si se reintenta.)
- **Solicitudes en CONFLICTO o RECHAZADO — cada cuánto y quién:** revisión **diaria** (y al cierre de cada jornada/feria) por el **responsable operativo/encargado de ventas** junto con soporte de datos; las `CONFLICTO` se resuelven con el cliente (aceptar nuevo precio → nueva solicitud) o se descartan, y las `RECHAZADO` se informan/reformulan. Se mantiene registro (motivo/estado) para auditoría.

3.c — Code review con IA (OpenCode, agente Plan, scripts de la cursada)

#	Unidad/archivo	Hallazgo	Decisión	Motivo / verificación
F1	U5 (seguridad) / `fn_autenticar`	Función `SECURITY DEFINER` sin `SET search_path` fijo (riesgo de search_path hijacking).	**Aceptado** (recomendado corregir)	Verificado en BD: `pg_get_functiondef` no muestra `SET search_path` y `pg_proc.proconfig` es NULL para `fn_autenticar`. Corrección: `ALTER FUNCTION ... SET search_path = public, pg_temp` (endurecimiento; no altera la lógica).
F2	TP3/TP5 (índices) / `idx_pedido_fecha` vs `idx_pedido_forma_fecha`	Sugerencia de la IA: eliminar `idx_pedido_fecha(fecha)` por redundancia con `idx_pedido_forma_fecha(forma_pago,fecha)`.	**Descartado con fundamento**	Verificado (`pg_indexes`): coexisten `idx_pedido_fecha(fecha)`, `idx_pedido_forma_fecha(forma_pago,fecha)`, etc. **Se descarta** el borrado porque el índice compuesto tiene `forma_pago` como columna líder y no beneficia igual a consultas que filtran **solo por rango de `fecha`** (caso TP3 consultas 2/3); `idx_pedido_fecha` cubre ese patrón. Mantener ambos evita sobre-optimizar para un caso rompiendo otro.
F3	TP3 (procedimientos) / `sp_registrar_pedido`	Repite la subconsulta de `precio` por ítem y no valida producto inexistente explícitamente: el precio queda `NULL` y falla por restricción.	**Aceptado**	Verificado en código (`pg_get_functiondef`: dos `(SELECT precio FROM producto WHERE id=...)`) y en BD: `BEGIN; CALL sp_registrar_pedido(966,'EFECTIVO',ARRAY[9999999]::bigint[],ARRAY[1]::int[]); ROLLBACK;` ⇒ `ERROR: valor nulo en "precio_unitario" viola not-null` (detalle con `precio/subtotal NULL`). Conteos PG sin cambios tras ROLLBACK (200401/601205). Mejora: resolver precio una vez por ítem y validar existencia con error claro.

Evidencias: scripts en `movil/` (`schema_sqlite.sql`, `pruebas_sqlite.sql`, `semaforo.sql`, `servidor_snapshot.csv`, `foodstore_movil.db`, `evidencia_*.txt`), spec en `docs/kiro/`, bitácora/DUIA en `docs/`. C1 comprobado offline; C7: cantidad 0 y producto inexistente (9999999) fallan sin filas nuevas.